Prompt Print: HackHarvard Build Plan

Oct 16, 2026 · @Nico

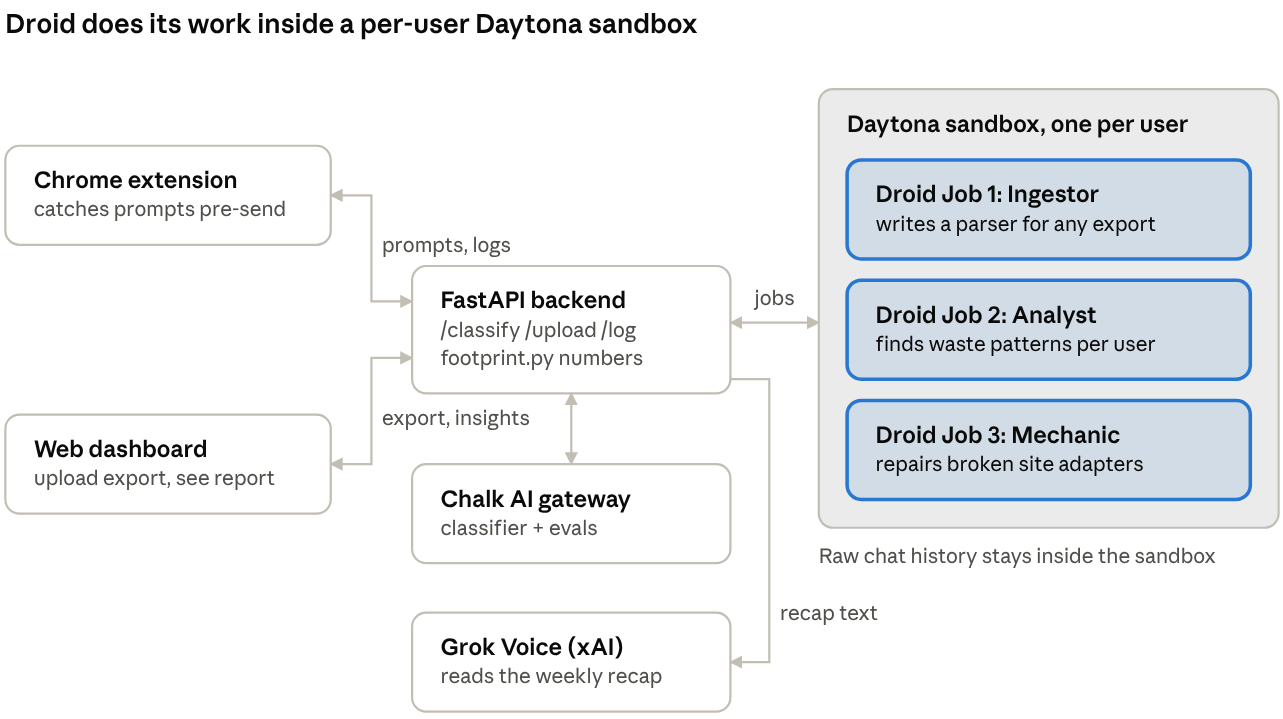
Pitch

Prompt Print tracks your AI usage, shows the energy, water, and CO_2 behind it, and coaches you before you send a wasteful prompt. Factory's Droid does the heavy lifting inside the running app: it writes the code that reads each user's chat history, analyzes their habits, and repairs the browser extension when ChatGPT changes its page.

One-line pitch: **"Everyone here used AI today. Nobody knows what it cost. We built the app that tells you, and helps you cut it."**

Prize	What wins it	Priority
Factory ($2,000 + credits)	Droid does three real jobs at runtime (see Factory section)	1
Daytona ($5,000 credits)	Every Droid run and every piece of private chat data lives in an isolated sandbox	2
Chalk ($1,500+)	Two evals with real numbers: parser accuracy and "small model is enough" quality	3
ASUS Sustainability (main track)	Cuts the footprint of a habit nearly everyone has, every day	3
SpaceXAI (keyboards)	Built in Cursor, Grok Voice recap, NASA satellite data on data center regions	4

Architecture



Prompt Print architecture · Droid's three jobs in Daytona

The backend never runs agent code itself: every Droid job runs in the user's sandbox and hands back only finished results.

Factory Droid: three jobs at runtime

The rule Factory judges on: Droid must do real work in the running product, not just help you write code. So Droid is the engine behind three features, each triggered by a user action or a failure, never by you.

Use the **Python Droid SDK** (`pip install droid-sdk`, import `droid_sdk`). It runs a local `droid` subprocess, so the `droid` CLI must be installed and authenticated with `FACTORY_API_KEY` wherever the SDK runs. That place is the Daytona sandbox (next section). Use `await run(...)` for one-shot jobs and `Session(...)` when a job needs follow-up turns.

Job 1: The Ingestor (writes a parser for any export)

- **Trigger:** user uploads a chat export (ChatGPT `conversations.json`, Claude export, Gemini Takeout, or a random CSV).

- **What Droid does:** inspects the file, writes `parser.py` that converts it to our fixed schema, writes a small test, runs it, fixes errors until the test passes.
- **Output:** `prompts.jsonl`, one line per exchange: `{ts, provider, model, kind (text/image/video), prompt_text, response_text, regenerated (bool)}`.
- **Why it needs an agent:** every provider's export is shaped differently and changes over time. Hard-coding one parser per provider breaks; Droid adapts.
- **Cache:** save each working parser keyed by a fingerprint of the file's structure, so the second upload of the same format skips Droid.

Job 2: The Analyst (finds this user's waste patterns)

- **Trigger:** after ingest, and weekly.
- **What Droid does:** writes and runs pandas analysis over `prompts.jsonl` to find user-specific patterns: repeated near-identical questions, regenerate loops, very long outputs for short asks, image or video generation spikes, time-of-day patterns.
- **Guardrail:** Droid never invents footprint numbers. It imports our fixed `footprint.py` module (see Footprint model) and only calls it.
- **Output:** `insights.json` with the top 3 coaching tips, each with evidence (counts, example prompts) and estimated savings.

Job 3: The Mechanic (self-healing extension adapters)

- **Trigger:** the browser extension fails to capture a prompt on ChatGPT, Claude, or Gemini (their page layout changed and our DOM selectors broke).
- **What Droid does:** receives the failing page snapshot, rewrites that site's adapter file (selectors for prompt box, send button, model name), runs it against the snapshot until it extracts the prompt correctly.
- **Output:** a new adapter JSON the extension downloads and hot-loads, no reinstall.
- **Why it wins:** it is a real operational problem every scraping product has, and the demo moment is strong: break a selector on purpose, watch Droid fix it live.

README requirement

Factory requires the README to explain where and how Droid is used. Add a section titled **"Where Droid works"** with the three jobs above, the trigger for each, and links to the code that calls the SDK. Write it Friday night, not Sunday morning.

Daytona: where Droid and private data live

Each user gets their own Daytona sandbox, and Droid runs inside it. Factory's own example app recommends isolating the Droid SDK in a separate container because the agent can run commands and edit files. That makes the Daytona integration necessary, not decorative.

Per-user flow (Python `daytona` SDK):

1. `sandbox = daytona.create(...)` from a prebuilt snapshot that already has Python, pandas, the `droid` CLI, `droid-sdk`, and our `footprint.py`. Building the snapshot once saves minutes per demo.
2. `sandbox.fs.upload_file(export_bytes, "/work/input/export.json")` uploads the user's chat export.
3. `sandbox.process.exec("python /work/jobs/ingest.py")` starts the worker script, which calls the Droid SDK for Job 1, then Job 2.
4. Stream logs back to the UI (judges see Droid writing code live).
5. `sandbox.fs.download_file("/work/output/insights.json")` returns results to the backend.
6. Delete or stop the sandbox afterwards (ephemeral sandboxes are an option). Say this out loud in the demo: **the raw chat history never touches our server's disk.**

Secrets: pass `FACTORY_API_KEY` and the Chalk gateway key into the sandbox as environment variables at creation, never baked into the snapshot.

Job 3 (Mechanic) gets its own short-lived sandbox per repair, with the page snapshot and the adapter file uploaded.

Pitch line for Daytona judges: "Untrusted, agent-written code running on private data is exactly the problem sandboxes solve."

Footprint model

Every number is a range (low to high), anchored to the two public per-prompt figures. This lives in one file, `footprint.py`, that Droid imports but never edits.

Anchors: Google measured the median Gemini Apps text prompt at 0.24 Wh, 0.03 gCO_2e, and 0.26 mL of water (May 2025 data). OpenAI's CEO stated an average ChatGPT query uses 0.34 Wh and about 0.000085 gallons (about 0.39 mL) of water. Neither counts model training.

Energy per text exchange, scaled by output length (longer answers cost more):

$$E = e_{\text{tok}} \cdot N_{\text{out}}, \qquad e_{\text{tok}} \in \left[\frac{0.24\ \text{Wh}}{N_{\text{med}}},\ \frac{0.34\ \text{Wh}}{N_{\text{med}}} \right]$$

N_out = output tokens of this response (count with `tiktoken`). N_med = median output tokens across your sample data. This assumes the published median prompt has a median-length answer; say that assumption out loud if asked.

Water:

$$W = E \cdot w, \qquad w \approx \frac{0.26\ \text{mL}}{0.24\ \text{Wh}} \approx 1.1\ \text{mL/Wh}$$

CO_2:

$$C = E \cdot c_{\text{grid}}$$

c_grid = carbon intensity (gCO_2/kWh) of the likely data center region, computed from EIA hourly generation by fuel for that grid region. Show it as "likely region," never as a fact.

Images and video: use a separate, much larger per-generation estimate taken from published research (start with Luccioni et al., "Power Hungry Processing," 2023, which measured image generation as far more energy-intensive than text). Look up and cite the exact figure before using it; do not guess.

Regenerations count as full extra exchanges. They are the easiest waste to cut and the most relatable stat in the demo.

Everyday equivalents for the UI: phone charges, seconds of TV (Google's own comparison: one median prompt is under 9 seconds of TV), water bottles.

Pre-send coaching extension

The extension changes behavior at the moment of use: it catches a prompt before it is sent and suggests a cheaper option. This is the feature that makes Prompt Print a daily habit instead of a weekly report.

How it works (Chrome, Manifest V3, content script):

1. A per-site **adapter** (JSON: selectors for prompt box, send button, model picker) tells the content script where things are on ChatGPT, Claude, and Gemini. Adapters are downloaded from our backend, so Droid's Mechanic can update them without a reinstall.
2. On Enter or Send, the script reads the prompt text and current model, then calls our backend `/classify`.
3. `/classify` sends the prompt to a small, fast model through **Chalk's AI gateway**, which returns one label: `small_ok`, `needs_big`, or `use_search` (a plain fact a search engine answers).
4. If the label suggests a cheaper option and the user is on a big model, show a small toast for about 3 seconds: "A smaller model can answer this. About 70% less energy. Switch / Send anyway." Never block the send.
5. Log every exchange (site, model, kind, prompt length, response length, regenerate clicks, whether the user took the suggestion) to the backend.

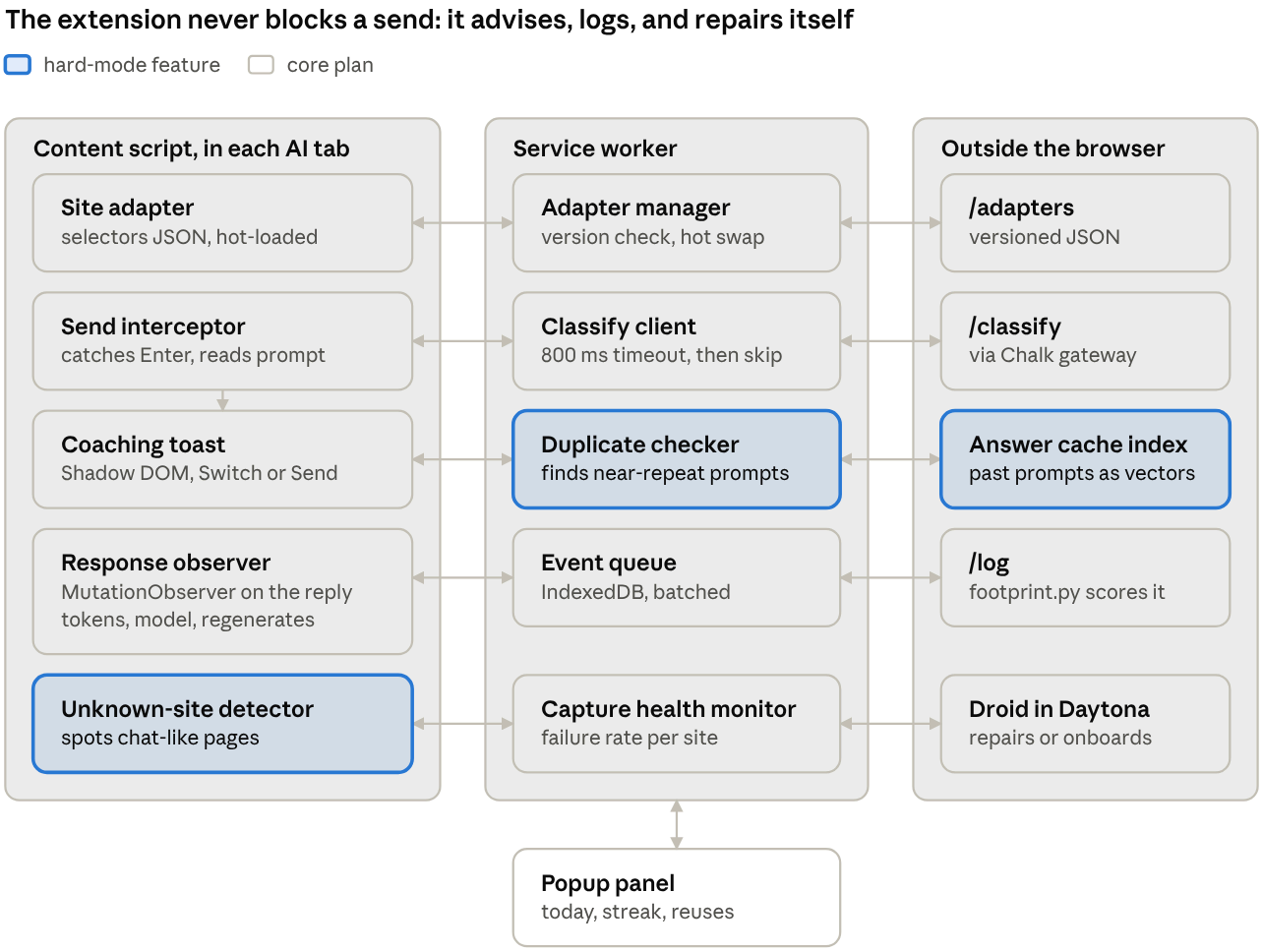
Latency budget: the toast must appear in under 1 second or people will hate it. If `/classify` is slow, send immediately and show the tip after.

Regenerate detection: count clicks on the site's regenerate button as extra exchanges. If it hits 3 on one answer, show "Try rewording instead of regenerating."

Scope cut: support ChatGPT first. Add Claude only if ChatGPT works by Saturday 2 PM.

The percent in the toast must come from `footprint.py` for the two models involved, not a hard-coded number.

Extension architecture



Send path: interceptor, classify (under 800 ms), toast, send. The reply is then observed and logged.

Extension architecture · content script, service worker, outside services

Each row is one job: the content script does it on the page, the service worker coordinates it, and an outside service finishes it. The adapter row is what Droid rewrites when a site breaks.

Space data and Grok

SpaceXAI is the lowest prize, so this gets the minimum that is still honest: one strong screen, one voice feature.

"Where your prompts run" screen (precomputed, not live):

- Pick 2 or 3 publicly known data center hubs (Northern Virginia's Loudoun County is the best known; confirm the others with a quick search).

- For each: a Sentinel-2 time-lapse of the area over about 5 years showing campuses being built, exported once from Copernicus Browser or Google Earth Engine as a GIF or image strip.
- Next to it: NASA GRACE groundwater trend for that region as a small line chart.
- Caption: "Your prompts likely ran in regions like this." Never claim we know the exact building.

Grok Voice (required for the SpaceXAI track, pick one):

- **Must have:** weekly recap read aloud with xAI's Text to Speech endpoint (`POST https://api.x.ai/v1/tts` , choose one of the 5 built-in voices, such as `eve`). The text comes from `insights.json` .
- **Stretch:** "Ask your report" voice agent using the realtime Voice Agent API (`wss://api.x.ai/v1/realtime?model=grok-voice-latest`) with a tool that reads `insights.json` , so a judge can ask "what was my most wasteful day?" out loud.

Grok Imagine: skip unless you have spare time Saturday night. If used: a shareable weekly card image.

Cursor: everyone codes in Cursor all weekend. Screenshot a few Cursor sessions for the Devpost write-up, since that track rewards heavy use.

Chalk evals

Chalk judges on impact, use of data, and how the agent is evaluated, so two evals with real numbers carry this prize. Route all model calls (classifier, small vs big models, the judge) through **Chalk's AI gateway**, so Chalk is in the stack, not bolted on. Confirm the exact eval API at the Chalk table Saturday morning (sponsor fair, 10 AM to 12 PM).

Eval 1: Is the coaching safe? (the important one)

1. Build a set of about 120 real-looking prompts: a third trivial, a third medium, a third hard (coding, math proofs, long analysis).
2. Run each through a small model and a big model.
3. An LLM judge compares the two answers and labels the small one `good_enough` or `worse` .
4. Run our `/classify` on the same prompts.
5. Report: (a) classifier accuracy against the judge's labels, (b) the rate where we said `small_ok` but the small answer was `worse` (the harmful mistake; keep it low), (c) energy saved if users followed every suggestion.

6. Hand-check about 20 judge labels so you can say the judge itself was spot-checked.

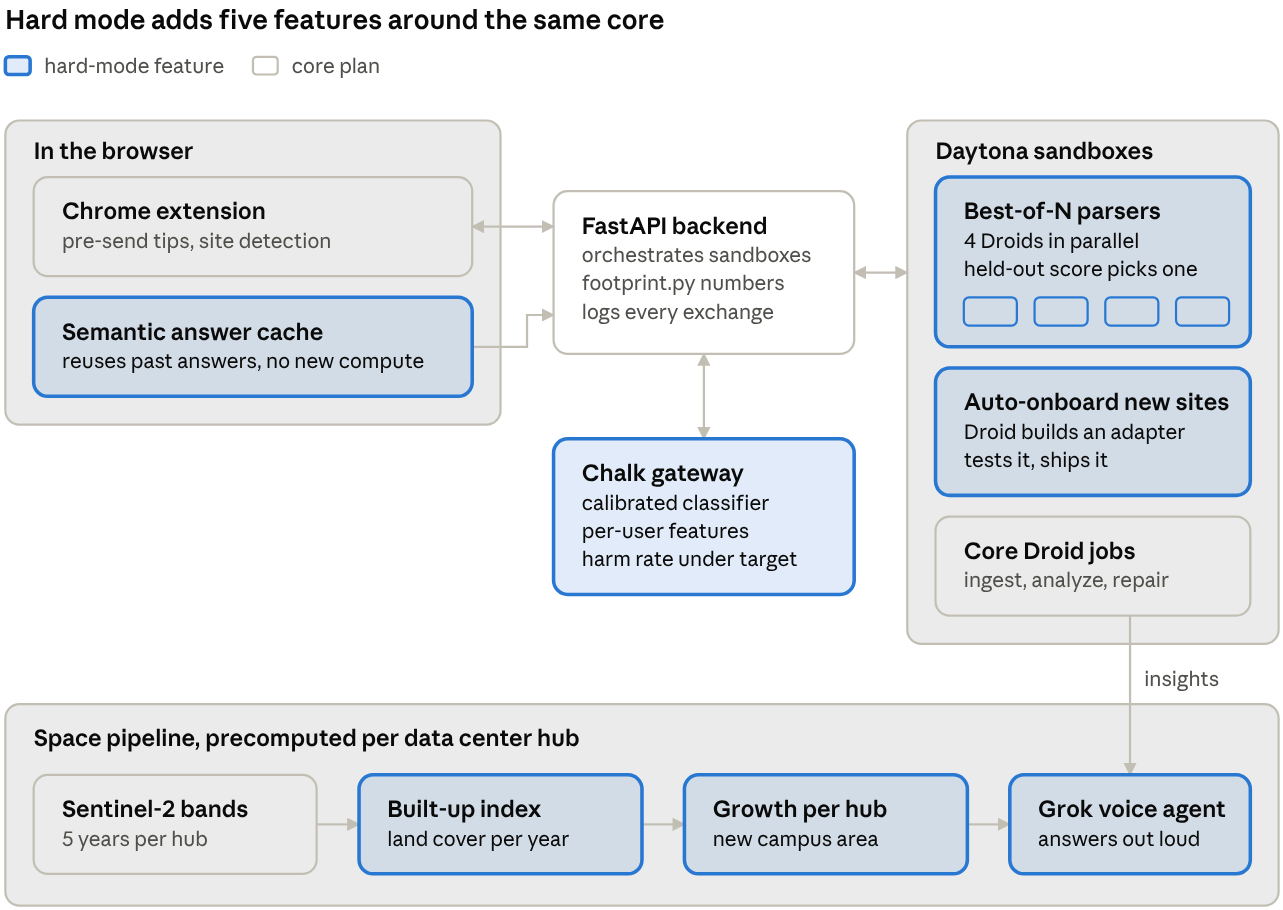
Eval 2: Does Droid's parser get the data right?

1. Hand-label one real export: the true number of exchanges, models, regenerations, and image generations.
2. Run the Ingestor (Job 1) on it 5 times from scratch.
3. Report field-level accuracy and how often Droid produced a working parser on the first try vs after fixes.

Show both as a small results table in the app and on Devpost. One honest number beats a vague claim, even if it is not perfect.

Hard mode: one hard feature per track

Build these only after the core plan works end to end (target: Saturday 2 PM). Build them in table order, since it follows prize value.



Hard mode architecture · five added features highlighted

Highlighted boxes are the hard-mode additions; everything else is the core plan they plug into.

Track	Hard feature	Why it's hard	Why judges care
Factory	**Auto-onboard any AI site.** The user opens an AI chat site we've never seen (Perplexity, Mistral). The extension detects a chat-like page, and Droid builds a working adapter from scratch, tests it on the live page snapshot, and ships it.	Droid has to work out an unknown page's structure with no hand-written code, then prove the result works.	Droid creates new product capabilities while the app runs. That is exactly the Factory brief.
Daytona	**Best-of-N parallel Droids.** For each new export format, start 4 sandboxes from the snapshot at once. Each Droid writes its own parser independently. Score every parser on a held-out slice of the file and keep the winner.	Parallel sandbox orchestration, scoring, and cleanup within a few seconds of each other.	It shows why fast, disposable sandboxes matter: many attempts at once, running untrusted code.

Track	Hard feature	Why it's hard	Why judges care
Chalk	**Risk-controlled coaching.** The classifier outputs a confidence. On Eval 1, plot accuracy against how often we give advice, and choose the threshold where the harmful-mistake rate stays under a target you set (for example 2%). Add per-user features (rolling regenerate rate, typical prompt length) through Chalk so the classifier adapts to each person.	Calibration plus a safety guarantee measured on held-out data, not an average accuracy number.	It turns "how it's evaluated" into a real safety claim, which is Chalk's judging criterion.
ASUS Sustainability	**Semantic answer cache.** Before a prompt is sent, compare it to the user's past prompts by meaning (embeddings). If they already asked essentially the same question and the answer isn't time-sensitive, the toast offers the old answer instead, so no new compute is used. Every reuse is logged as an avoided exchange.	Fast similarity search under the 800 ms budget, a match threshold tuned on labeled pairs so wrong matches stay rare, and detecting time-sensitive questions ("today's weather") that must never be reused.	The most sustainable prompt is the one never sent. It cuts real compute instead of only reporting it, with no hardware needed.

Track	Hard feature	Why it's hard	Why judges care
SpaceXAI	**Automatic data center growth detection.** Instead of hand-picked time-lapses, compute a built-up index from Sentinel-2 bands over time for each hub and measure how much land the campuses cover each year. Add a realtime Grok voice agent that answers questions using these results and the user's own data.	Satellite change detection plus a realtime voice agent with tool calls.	Real space data analysis instead of decoration, plus Grok Voice as an interactive feature.

Honest scope check: a 4-person team in about 30 working hours can realistically finish the core plan plus one or two of these. Finishing two well beats five half-built features at judging. The Factory and Daytona rows protect the two biggest prizes, so start there.

Stack and repo layout

Python on the backend, because the Droid, Daytona, and Chalk SDKs are all Python.

Part	Tool
Backend API	FastAPI
Web dashboard	React + Vite, Recharts for charts
Extension	Chrome Manifest V3, plain TypeScript
Agent	Factory Droid SDK (Python), inside Daytona
Sandboxes	Daytona Python SDK, one prebuilt snapshot
Model calls + evals	Chalk AI gateway
Voice	xAI TTS (realtime voice as stretch)
Storage	SQLite (enough for a hackathon)

Part	Tool
Token counts	tiktoken

```
prompt-print/
  backend/
    main.py            # FastAPI: /upload, /classify, /log, /insights,
/adapters
    sandbox.py         # Daytona: create, upload, exec, download
    footprint.py       # the only place numbers come from
    voice.py           # xAI TTS recap
  sandbox_jobs/        # copied into the Daytona snapshot
    ingest.py          # Droid Job 1
    analyze.py         # Droid Job 2
    repair_adapter.py  # Droid Job 3
    schema.json        # fixed prompts.jsonl schema
  extension/
    content.ts  background.ts  adapters/chatgpt.json
  web/                 # dashboard
  evals/
    prompts_120.jsonl  run_eval1.py  run_eval2.py
  space/               # precomputed time-lapses + GRACE charts
  README.md            # includes "Where Droid works"
```

Team roles and timeline

Written for 4 people; with 3, merge Person C into A and B. The venue closes overnight, so plan real work time at wherever you are staying.

Person	Owns	Prize it protects
A	Droid jobs + Daytona sandbox flow	Factory, Daytona
B	Chrome extension + `/classify` + adapters	Factory (Job 3), ASUS
C	Dashboard, voice recap, space screen	SpaceXAI, demo polish
D	`footprint.py`, both evals, README, Devpost	Chalk, Factory README

When	Goal (must be true by the end)
Fri 5 to 9 PM	Accounts and API keys for Factory, Daytona, Chalk, xAI. `droid` running inside one Daytona sandbox. Sample ChatGPT export in hand. Footprint formulas agreed.
Fri night	Job 1 turns a real export into `prompts.jsonl`. Extension logs one ChatGPT prompt to the backend. README "Where Droid works" drafted.
Sat 10 AM to 12 PM	Talk to Chalk, Daytona, and Factory sponsors at the fair. Confirm Chalk eval API.
Sat 12 to 6 PM	Job 2 insights on the dashboard. Pre-send toast working. Eval 1 dataset built. Voice recap plays. Attend Daytona workshop (2:30 PM) if stuck.
Sat 6 PM to midnight	Job 3 Mechanic working on a deliberately broken selector. Both evals run, numbers in the app. Space screen done. **Feature freeze at midnight.**
Sun 12 to 6 AM	Bug fixes only. Saved fallback outputs for every live step. Record a backup demo video. Rehearse the 3-minute demo 3 times.
Sun by 8 AM	Devpost submitted with video, README, eval table, Cursor screenshots.

Demo plan (3 minutes)

Each Droid job gets its own visible moment, because Factory is the prize to win.

1. **Hook (20 s):** "Raise your hand if you used AI today." Then a judge guesses which of three prompts costs the most: a quick question, an essay, an AI image.
2. **Live coaching (30 s):** type a trivial question into ChatGPT on a big model. The toast appears suggesting a smaller model. Click switch.
3. **Droid Job 1 + 2 (50 s):** upload a teammate's real chat export. Show the Daytona log stream as Droid writes and tests the parser, then the dashboard with weekly totals and the funniest stat (most-regenerated prompt).

4. **Droid Job 3 (30 s):** flip a switch that breaks the ChatGPT selector. The extension reports failure, Droid repairs the adapter in a sandbox, capture works again. This is the Factory moment.
5. **Proof (25 s):** Chalk eval table: classifier accuracy and the harmful-mistake rate.
6. **Close (25 s):** play the Grok Voice recap, show the "where your prompts run" time-lapse, and finish with the physical comparison (a real water bottle on the table).

Fallbacks: every live step has a saved output ready to show if the Wi-Fi, a sandbox, or Droid is slow. One person talks, one person drives the laptop.

Risks and what to cut

Risk	What to do
`droid` CLI won't install or authenticate inside Daytona	Test this first, Friday 5 PM. If it fails by 8 PM, run Droid on the backend machine and keep Daytona for executing the code Droid writes. Ask Factory and Daytona sponsors immediately.
Droid is slow (minutes per job)	Cache parsers by file structure. Precompute the demo export once and show the saved run if needed.
ChatGPT page changes or blocks the extension	Job 3 is literally built for this. Keep a saved page snapshot for the demo.
Energy numbers challenged by a judge	Show ranges, name the Google and OpenAI sources, state the median-length assumption.
Eval 1 takes too long to run	Shrink to 60 prompts. A smaller honest eval beats none.
Running out of time	Cut in this order: Grok Imagine, realtime voice agent, Claude and Gemini adapters, space time-lapse (keep one static image). Never cut Job 3 or Eval 1.

Open question for the team: who has a real ChatGPT export with a few weeks of history they are comfortable showing judges? Get it tonight.

Sources

- HackHarvard 2026 challenges
- Factory Droid Python SDK docs
- droid-sdk on PyPI
- Factory simple-chatapp example (isolation advice)
- Daytona Python SDK
- Chalk agent sandboxes
- Chalk Compute
- xAI Voice docs
- Google: measuring the environmental impact of AI inference
- DatacenterDynamics on Google and OpenAI per-prompt figures